courseCS.WordPress.com

Lesson-3-Data-Types

Download Lesson

Previous Lesson Next Lesson

3.1 Types of Data

In programming, data types is an important concept. To be able to operate on variables,. it is important to know something about the type. Without data types, a computer cannot safely work on the data.

JavaScript has 8 Datatypes

1. Number
2. BigInt
3. String
4. Boolean
5. Undefined
6. Null
7. Symbol
8. Object

3.1.1 Number

Numbers can be either integer, float, real or any other numeric type data.

Example Code 1:

In following example, we used “let” as we discussed in previous lesson that modern approach to declare variable is based on “const” and “let”. In this example, we just declared some numeric variables and then displayed their values.

```
// Numbers:
let length = 16;
let weight = 7.5;
//Exponential Notation
let y = 123e5;    // result will be 12300000
let z = 123e-5;   // result will be 0.00123

//displaying values of variables
console.log(length)
console.log(weight)
console.log(y)
console.log(z)
```

Most programming languages have many number types that are listed below but **JavaScript numbers are always one type that is double (64-bit floating point).**

- Whole numbers (integers)
  - byte (8-bit), short (16-bit), int (32-bit), long (64-bit)
- Real numbers (floating-point)
  - float (32-bit), double (64-bit)

Note: JavaScript integers are only accurate up to 15 digits

3.1.2 BigInt

All JavaScript numbers are stored in a a 64-bit floating-point format. JavaScript BigInt is a new datatype (**ES2020**) that can be used to store integer values that are too big to be represented by a normal JavaScript Number.

In JavaScript, all numbers are stored in a 64-bit floating-point format (IEEE 754 standard). With this standard, large integer cannot be exactly represented and will be rounded. Because of this, JavaScript can only safely represent integers from **+9007199254740991** to **-9007199254740991**. In other words, range is **+(2^{53}-1)** to **-(2^{53}-1)**. Integer values outside this range lose precision.

Example Code 2:

In below example, we discussed two method to declare BigInt type variable. One is by calling BitInt() and second is by appending “n” at end of the integer.

```
// BigInt by 1st method
let x = BigInt("123456789012345678901234567890");
// BigInt by second method
let y = 9999999999999999n //appendring n to the end of an integer

//displaying values of variables
console.log(x)
console.log(y)
```

3.1.2.1 Important Instruction About BigInt

You must know followings related to BigInt.

- Operators that can be used on a JavaScript `Number` can also be used on a `BigInt`.
- Arithmetic between a `BigInt` and a `Number` is not allowed. Type-conversion can solve the issue but it may lose some information from BigInt Data.
- A `BigInt` can not have decimals
- `BigInt` is supported in all browsers since September 2020

3.1.3 String

String is either one or combination of multiple characters.

Example Code 3:

In following piece of code, we just declared some string variables and then displayed their values. String data can be declared by using either double or single quotation marks.

```
// String with double quotation marks
let color = "Yellow";
// Strning with single quotation marks
let lastName = 'Johnson';

//displaying values of variables
console.log(color)
console.log(lastName)
```

3.1.4 Boolean

Boolean type variable can have two values, either TRUE or FALSE.

Example Code 4:

In following piece of code, we just declared some Boolean variables and then displayed their values.

```
// Booleans
let x = true;
let y = false;

//displaying values of variables
console.log(x)
console.log(y)
```

3.1.5 Undefined

If a variable is declared but not initialized, then the variable will contain undefined data.

Example Code 5:

Following piece of code doesn't have some error. Output of this code is "undefined".

```
let z

//displaying values of variables
console.log("Value of z is: ",z)
```

3.1.6 Null

If a variable is declared but initialized with some value that is nothing, then the variable will contain null data. Actually, null data is considered into string, therefore we used quotation marks during variable initialization.

Example Code 6:

Following piece of code doesn't have some error. Output of this code is nothing.

```
let z = "" //null data with double quotation marks
let x = '' //null data with single quotation marks

//displaying values of variables
console.log("Value of z is: ",z)
console.log("Value of x is: ",x)
```

3.1.7 Symbol

If a variable is declared but initialized with some symbol, then this type of data is symbolic in the variable. Actually, symbol data is considered into string, therefore we used quotation marks during variable initialization. Output of this code is a symbol.

Example Code 7:

```
let z = "$" //null data with double quotation marks
let x = '#' //null data with single quotation marks

//displaying values of variables
console.log("Value of z is: ",z)
console.log("Value of x is: ",x)
```

3.1.8 Object

This is last data type that holds three different type of data-formats. They are explained in next section.

3.2 Types of Object's Data

The object data type can contain:

1. An object
2. An array
3. A date

3.2.1 Object

JavaScript objects are written with curly braces {}. Object properties are written as name: value pairs, separated by commas.

Example Code 8:

In following piece of code, the object (person) has 4 properties: firstName, lastName, age, and eyeColor. It is noted that "const" is used for declaring objects we discussed above. Moreover, curly braces are used for declaration.

```
const person = {
    firstName : "John",
    lastName  : "Doe",
    age     : 50,
    eyeColor  : "blue"
  };
  console.log("person age: ", person.age)
  console.log("person first name: ", person.firstName)
```

Example Code 9:

In above example, object is declared in a proper way. The object can also be declared in one line as you can see in below example.

```
const person = {firstName : "John", lastName  : "Doe", age     : 50, eyeColor  : "b
  console.log("person age: ", person.age)
  console.log("person first name: ", person.firstName)

```

Example Code 10:

In above two examples, object's properties are accessed by using dot operator "." but we can also use square braces and double quotation marks for the same purpose. See below example, in which above code is written but square braces and double quotation marks are used to display age and firstName of the object.

```
const person = {
  firstName : "John",
  lastName  : "Doe",
  age     : 50,
  eyeColor  : "blue"
};
console.log("person age: ", person["age"])
console.log("person first name: ", person["firstName"])
```

Note: You will learn more about object later. We allocated a detailed lesson for object's concepts. There are many more concepts related to objects.

3.2.2 Array

JavaScript arrays are written with square brackets. Array items are separated by commas.

Example Code 11:

The following code declares (creates) an array called `cars`, containing three items (car names). Array's elements can be accessed by using index number of elements. The first item has index number is 0, second has 1, and so on. Line 2 to 4 show starting, first and second element of "cars" array. It is noted that array is declared by using "const" as we discussed in previous lesson.

```
const cars = ["Saab", "Volvo", "BMW"]
console.log(cars[0])
console.log(cars[1])
console.log(cars[2])
```

Example Code 12:

In following code, we declared array in multiple lines. This method is only for increasing readability of code. Output is same as your saw for previous code.

```
const cars = [
  "Saab",
  "Volvo",
  "BMW"
]
console.log(cars[0])
console.log(cars[1])
console.log(cars[2])
```

Example Code 13:

Above codes can also be write in following way. In below mentioned method, we initially declared "cars" array as blank and then move values in array at its index numbers.

```
1  const cars = []
2  cars[0]= "Saab"
3  cars[1]= "Volvo"
4  cars[2]= "BMW"
5  console.log(cars[0])
6  console.log(cars[1])
7  console.log(cars[2])
```

In above code, you can change location of line 2, 3 and 4. Because it is not necessary to initialize elements in a sequence.

Note: Arrays are a special type of objects. The “typeof” operator in JavaScript returns “object” for arrays. But, JavaScript arrays are best described as arrays. Arrays use **numbers** to access its “elements”. While, Objects use **names** to access its “members”.

3.2.3 Date

Date objects are created with the `new Date()` constructor.

Example Code 14:

```
1  //const is used to store value
2  const current_date = new Date()
3
4  //display the obtained-value by above line
5  console.log(current_date)
```

In above code, `new Date()` creates a date object with the **current date and time**. You can save the obtained value in “const” type variable as you can see in above code.

Exercises

Solve following exercises to enhance your learnt concepts related to this lesson.

1. Declare a variable called “name”, initialize it with your name and then display your name
2. Declare a variable called “age”, initialize it with your age and then display your age
3. Create an array called fruits having names of your favorite fruits and display each element without using loop
4. Declare a variable called currentYear, initialize it with the current year (e.g., 2023), reassign the value of currentYear to the next year and display it.
5. Create a variable called isRainyDay, set it to true or false based on the weather and display its value
6. Create an object called personInfo with properties like name, age, and city. Display all data of the object.

▼ Solution for Exercise 1

```
let myName = "M Abo Bakar Aslam"
console.log(myName)
```

▼ Solution for Exercise 2

```
let myAge = 32
console.log(myAge)
```

▼ Solution for Exercise 3

```
const fruit = ["apple", "mango", "orange", "banana", "graps", "pineapple"]
console.log(fruit[0])
console.log(fruit[1])
console.log(fruit[2])
console.log(fruit[3])
console.log(fruit[4])
console.log(fruit[5])
```

▼ Solution for Exercise 4

```
let currentYear = 2023
currentYear = 2024
console.log(currentYear)
```

▼ Solution for Exercise 5

```
let isRainyDay = true
if (isRainyDay == true) {
    console.log("Rainy Day")
}else{
    console.log("Not Rainy Day")
}
```

▼ Solution for Exercise 6

```
const person = {
    name: "M Abo Bakar Aslam",
    age: 32,
    city: "Lahore"
}
console.log(person)
console.log(person.name)
console.log(person.age)
console.log(person.city)
```